

Birthday Cake Candles Algorithms

Algorithm 1: Non-Recursive

Algorithm birthdayCakeCandels(CANDLES, x)

```
{
    max_height ← CANDLES[0]
    count ← 0

    for i ← 0 to x - 1 do
        if CANDLES[i] > max_height then
            max_height ← CANDLES[i]
            count ← 1
        else if CANDLES[i] = max_height then
            count ← count + 1

    return count
}
```

Time Complexity Analysis

$$\begin{aligned} T(n) &= O(1) + \sum_{i=0}^{n-1} O(1) \\ &= O(1) + n * O(1) \\ &= O(1) + O(n) \\ &= O(n) \end{aligned}$$

Final Time Complexity: $T(n) = O(n)$

Algorithm 2: Recursive

Algorithm brithdayCakeCandels(CANDLES, x, index, max_height, count)

```
{
    if index == x then
        return count

    if CANDLES[index] > max_height then
        return brithdayCakeCandels(CANDLES, x, index + 1, CANDLES[index], 1)

    else if CANDLES[index] = max_height then
        return brithdayCakeCandels(CANDLES, x, index + 1, max_height, count + 1)

    else
```

```

    return birthdayCakeCandels(CANDLES, x, index + 1, max_height, count)
}

```

Time Complexity Analysis

$$\begin{aligned}
 T(n) &= T(n-1) + O(1) \\
 &= [T(n-2) + O(1)] + O(1) \\
 &= T(n-2) + 2O(1) \\
 &= [T(n-3) + O(1)] + 2O(1) \\
 &= T(n-3) + 3O(1)
 \end{aligned}$$

In general:

$$T(n) = T(n-k) + kO(1)$$

When $k = n$:

$$\begin{aligned}
 T(n) &= T(0) + nO(1) \\
 &= O(1) + n * O(1) \\
 &= O(n)
 \end{aligned}$$

Final Time Complexity: $T(n) = O(n)$

Comparison Between Non-Recursive and Recursive Algorithms

Aspect	Non-Recursive	Recursive
Approach	Iterative loop	Recursive function
Extra Memory	No extra memory used	Call stack used (recursive overhead)
Ease of Debugging	Easy to debug	Harder to trace due to recursive calls
Time Complexity	$O(n)$	$O(n)$
Suitability	Better for large inputs	Risk of stack overflow in large inputs